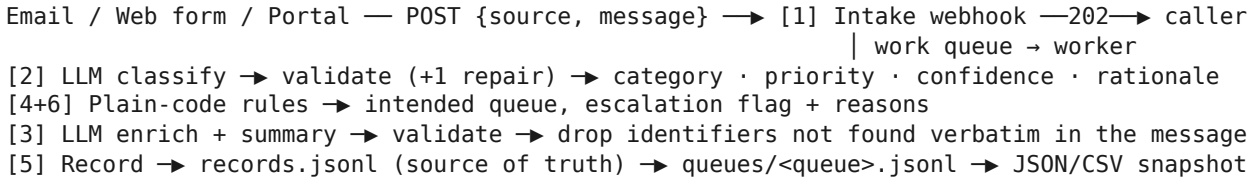
ArcVault Intake & Triage: Architecture Write-Up

1. System design



- **Trigger:** an HTTP webhook starts the pipeline on every new message. The Python service replies `202 Accepted` at once and processes in the background, so a slow LLM never times out the sender.
- **Two implementations, one set of prompts:** a dependency-free Python service (`intake_server.py`, with a live demo console at `localhost:8080`) and an n8n workflow generated from the same `prompts.py`. Both were run end to end with the real model.
- **Where state lives:** an append-only `records.jsonl` is the source of truth. Each team gets its own queue file, and a JSON/CSV snapshot is rebuilt for people to read. `record_id` is a hash of source and message, so a re-delivered message is skipped before any LLM call.
- **Why two LLM calls:** classification first, then enrichment that already knows the category and team, so the summary ends with an action for the right team. Priority is deliberately *not* passed to enrichment: when it was, the model quoted "Priority: High" back as urgency evidence.
- **Model:** `openai/gpt-oss-120b` on Groq's free tier. It's free, fast (about 2.4 s per message), supports JSON mode and has open weights, so it could be self-hosted later. The client is provider-agnostic; switching from the retired Llama 3.3 70B took one config value.
- **Results:** 5/5 correct on category, priority, queue, escalation and identifiers, in both versions (`output/eval_report.md`).

2. Routing logic

Category	Queue	Why
Bug Report	Engineering	Needs a code fix; engineering owns defects
Incident/Outage	Engineering On-Call	Needs a response now, not a backlog slot
Feature Request	Product	Product prioritises new capabilities
Billing Issue	Billing	Finance has invoice and contract access
Technical Question	Technical Support	"Is X supported?", often pre-sales (the Okta SSO question)

The LLM proposes labels and a lookup table decides the queue. Routing is therefore **auditable** (every record keeps `intended_queue`), **tested** (14 unit tests, no API calls) and **changeable by operations** without touching prompts. The low-confidence fallback is the escalation queue below.

3. Escalation logic

A record goes to **Human Escalation** if any rule fires. Every rule that fires is listed in `escalation_reasons`, and the record keeps its intended queue so a reviewer can confirm it in one click.

1. **Confidence < 0.70:** the brief's threshold. If the model is unsure, a person checks before the ticket lands in the wrong queue, which is ArcVault's original problem.
2. **Category = Incident/Outage:** outages need a human owner, a status-page update and customer comms.
3. **Keywords in the raw text:** `outage`, `is/went down`, `down for all`, `multiple/all users`, security words (`breach`) and legal words (`lawsuit`, `chargeback`). Word-boundary regexes avoid false matches

such as “download”. These rules don’t depend on the model, so a misclassified outage is still caught.

4. **Billing error > $500.** *Assumption:* “error” means the disputed difference, not the invoice total. Sample 3 is $1,240 − $980 = **$260**, so it goes to Billing without escalation; a $3,000 edge case escalates. Switching to the invoice total is a one-line change, and I’d confirm the intent with ArcVault.
5. **Processing failure:** invalid output after one repair attempt, or an API error, goes to a human. Nothing is silently dropped or guessed.

On the samples, only sample 5 escalates, for two independent reasons: the category and the “multiple users” keyword.

Known limits, shown honestly:

- **Confidence isn’t calibrated.** Self-reported confidence sits around 0.97 for everything, which is why rules 2–3 don’t rely on it.
- **Two requests in one message go to one team.** A vague “something’s broken + send my invoice” scored 0.78 and went only to Engineering.
- **Prompt injection was resisted.** The customer text is wrapped in tags and treated as data, not instructions.

4. At production scale

- **Reliability:** use a durable queue (SQS or Redis Streams) with a dead-letter queue, and store records in Postgres. Deliver to Jira or Zendesk with retries, using `record_id` as the idempotency key.
- **Rate limits:** I hit this in testing (Groq’s free tier allows 8k tokens/minute). The client now honours `Retry-After`. Production needs a token-bucket limiter and a paid tier.
- **Cost:** a small 8B model classifies first, and only low-confidence messages go to a large model. At these prices a ticket costs fractions of a cent.
- **Latency:** use provider structured outputs to remove repair round trips, set per-call timeouts, and add a circuit breaker that routes everything to people if the LLM provider is down.
- **Quality:** log the model, prompt version and the reviewer’s final decision. That log becomes labelled data for calibrating the threshold and tracking accuracy per category. Prompt changes must pass `eval/evaluate.py` in CI.

5. Phase 2 (another week)

1. **Measured confidence:** self-consistency or logprobs, calibrated on real tickets.
2. **Multi-intent splitting:** one message becomes linked records for each team.
3. **Review actions in the console:** approve or override escalations, and feed each override back into the eval set.
4. **Context lookups:** the account’s plan and value, the invoice against the contract, and the status page. Priority can then reflect customer value, and duplicate outage reports can attach to one existing incident.